tarea5 -Estructuras de Datos, Divide y Vencerás

Análisis y Diseño de Algoritmos — Maestría PCIC (Semestre 2027-1)

Tlacaelel Jaime Flores Villaseñor Victor Rosales Jaimes

2026-11-08

1 Ejercicio 1. Heap binario máximo y heapsort

1.1 Enunciado

Considera los algoritmos de heap binario máximo y heapsort que aparecen en las diapositivas utilizadas en clase.

1.1.1 Inciso a. Propiedad de `sink`

Demuestra la siguiente propiedad de la función `sink` (diapositiva 20): si al llamar `sink(A, k, n)`, cada uno de los subárboles con raíces $A[2k]$ y $A[2k+1]$, respectivamente, cumple con la propiedad heap-max (es decir, el entero en cada vértice del árbol es mayor o igual que los enteros en sus hijos), entonces al finalizar `sink`, el subárbol con raíz $A[k]$ cumple con la propiedad heap-max.

1.2 Solución del inciso a

1.2.1 Funcionamiento de `sink`

1.2.2 Demostración

1.3 Enunciado del inciso b

Demuestra la siguiente invariante de ciclo de la fase de construcción de heapsort (ciclo de la diapositiva 42): al inicio del ciclo, para todo $k+1 \leq \ell \leq n$, se cumple que el subárbol con raíz en $A[\ell]$ cumple con la propiedad heap-max.

1.4 Solución del inciso b

1.4.1 Inicialización

1.4.2 Mantenimiento

1.4.3 Conclusión al terminar

2 Ejercicio 2. Emparejamiento de estaciones en un árbol

2.1 Enunciado

Sea T un árbol binario enraizado. Todos los vértices de T tienen un atributo binario `flag`, y solo para un número par de ellos, a los que llamaremos **estaciones**, su atributo es `true`.

Se busca emparejar las estaciones de forma tal que en T existan caminos disjuntos en aristas que conecten las parejas de estaciones. Los caminos no pueden compartir aristas, aunque sí pueden compartir vértices.

Diseña un algoritmo tipo divide y vencerás de tiempo $O(n)$ que imprima parejas de estaciones con la propiedad descrita, donde n es el número de vértices de T. Toda estación debe quedar emparejada.

Puedes suponer que existe una instrucción `Print(s)` que imprime la cadena `s` y toma tiempo $O(1)$. La raíz r de T puede ser parte de la entrada.

El árbol está representado como una lista de adyacencia. Para cada vértice v, left(v) y right(v) denotan sus hijos izquierdo y derecho, y $\pi(v)$ denota su padre. Si alguno no existe, su valor es `NONE`.

Explica la idea del algoritmo y da pruebas de terminación, corrección, tiempo y espacio. ¿Existen entradas para las que tu algoritmo corre en tiempo $O(\log n)$? Argumenta tu respuesta.

2.2 Idea del algoritmo

2.3 Pseudocódigo

2.4 Prueba de terminación

2.5 Prueba de corrección

2.5.1 Propiedad que se demostrará

2.5.2 Caso base

2.5.3 Paso inductivo

2.5.4 Conclusión para el árbol completo

2.6 Tiempo de ejecución

2.7 Espacio utilizado

2.8 ¿Existen entradas con tiempo $O(\log n)$?

3 Ejercicio 3. Reconstrucción de un árbol binario

3.1 Enunciado

Considera árboles binarios enraizados, representados como en el ejercicio anterior. En un recorrido se realiza una exploración tipo DFS desde la raíz y se produce una secuencia en la que cada vértice aparece una y solo una vez. Las siguientes políticas se aplican recursivamente:

- **En orden:** subárbol izquierdo, raíz y subárbol derecho.
- **Preorden:** raíz, subárbol izquierdo y subárbol derecho.
- **Postorden:** subárbol izquierdo, subárbol derecho y raíz.

Se reciben un entero n y dos arreglos de longitud n, EO y PO, que contienen los recorridos en orden y en preorden, respectivamente, de un árbol binario enraizado T. El árbol no tiene por qué ser completo ni balanceado.

Diseña un algoritmo tipo divide y vencerás de tiempo $O(n^2)$ que produzca T a partir de n, EO y PO. Explica su idea y da pruebas de terminación, corrección, tiempo y espacio.

¿Existen entradas para las que tu algoritmo corre en tiempo $O(n \log n)$? ¿Y $O(n)$? ¿Cómo modificarías tu algoritmo si PO tuviera el recorrido en postorden en lugar de preorden? Argumenta tus respuestas.

3.2 Idea del algoritmo

3.3 Pseudocódigo

3.4 Prueba de terminación

3.5 Prueba de corrección

3.5.1 Caso base

3.5.2 Hipótesis inductiva

3.5.3 Paso inductivo

3.6 Tiempo de ejecución

3.7 Espacio utilizado

3.8 Entradas con tiempo $O(n \log n)$

3.9 Entradas con tiempo $O(n)$

3.10 Modificación para usar postorden

3.10.1 Cambios en la idea del algoritmo

3.10.2 Pseudocódigo o cambios al pseudocódigo original

3.10.3 Justificación

4 Ejercicio 4. Consultas de rutas a gran escala

4.1 Enunciado

Ve el video indicado en la tarea y explica con tus propias palabras, en **no más de una página**, por qué el algoritmo de Dijkstra no es lo suficientemente rápido en sistemas del tamaño de empresas como Google y cómo puede modificarse para responder con rapidez a pesar de los grandes volúmenes de peticiones que reciben.

4.2 Explicación